

Lab

Name: Arden Diago Reg No:

Class: MCA - A

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
```

```
In [2]: DATA1='/home/arden/Coding/ML-Class-Work/data/city_day.csv'
DATA2='/home/arden/Coding/ML-Class-Work/data/crop_production.csv'
```

```
In [3]: def display_details(title, d1, d2):
        print(f" *** {title} ***")
        print(f"Data 1:\n{d1}\n\nData 2:\n{d2}")
```

```
In [4]: df1 = pd.read_csv(DATA1)
df2 = pd.read_csv(DATA2)

# print(f"* Data 1\n\n{df1.head()}\n\n* Data 2\n\n{df2.head()}")
display_details("Displaying the first 5 rows of the dataset", df1.head(),
```

*** Displaying the first 5 rows of the dataset ***

Data 1:

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	S02
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33

	03	Benzene	Toluene	Xylene	AQI	AQI_Bucket
0	133.36	0.00	0.02	0.00	NaN	NaN
1	34.06	3.68	5.50	3.77	NaN	NaN
2	30.70	6.80	16.40	2.25	NaN	NaN
3	36.08	4.43	10.14	1.00	NaN	NaN
4	39.31	7.01	18.89	2.78	NaN	NaN

Data 2:

	State_Name	District_Name	Crop_Year	Season \
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year

	Crop	Area	Production
0	Arecanut	1254.0	2000.0
1	Other Kharif pulses	2.0	1.0
2	Rice	102.0	321.0
3	Banana	176.0	641.0
4	Cashewnut	720.0	165.0

Task — Dataset Inspection and Profiling

For each dataset:

-
- Identify the number of rows and columns
- Examine column names and data types
- Identify missing values
- Detect duplicate records
- Review summary statistics
- Inspect categorical values and possible inconsistencies

Analyze the dataset structure

```
In [5]: # info of dataset 1
df1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   City            29531 non-null  str    
 1   Date            29531 non-null  str    
 2   PM2.5           24933 non-null  float64
 3   PM10           18391 non-null  float64
 4   NO              25949 non-null  float64
 5   NO2            25946 non-null  float64
 6   NOx            25346 non-null  float64
 7   NH3            19203 non-null  float64
 8   CO             27472 non-null  float64
 9   SO2            25677 non-null  float64
10   O3             25509 non-null  float64
11   Benzene        23908 non-null  float64
12   Toluene        21490 non-null  float64
13   Xylene         11422 non-null  float64
14   AQI            24850 non-null  float64
15   AQI_Bucket     24850 non-null  str    
dtypes: float64(13), str(3)
memory usage: 3.6 MB
```

```
In [6]: # info of dataset 2
df2.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   State_Name      246091 non-null  str
1   District_Name   246091 non-null  str
2   Crop_Year       246091 non-null  int64
3   Season         246091 non-null  str
4   Crop            246091 non-null  str
5   Area           246091 non-null  float64
6   Production      242361 non-null  float64
dtypes: float64(2), int64(1), str(4)
memory usage: 13.1 MB

```

```
In [7]: display_details("Missing Values", df1.isna().sum(), df2.isna().sum())
```

```
*** Missing Values ***
```

```
Data 1:
```

```

City          0
Date          0
PM2.5        4598
PM10         11140
NO           3582
NO2          3585
NOx          4185
NH3          10328
CO           2059
SO2          3854
O3           4022
Benzene       5623
Toluene       8041
Xylene       18109
AQI           4681
AQI_Bucket    4681
dtype: int64

```

```
Data 2:
```

```

State_Name      0
District_Name   0
Crop_Year       0
Season          0
Crop            0
Area            0
Production      3730
dtype: int64

```

Getting Categorical Values insights

```
In [8]: def categorical_insights(column_name, d):
        print(f"Unique Values: \n{d[column_name].unique()} \n\nNo of rows:
```

```
In [9]: categorical_insights("AQI_Bucket", df1)
```

```
Unique Values:
<StringArray>
[nan, 'Poor', 'Very Poor', 'Severe', 'Moderate', 'Satisfactory', 'Good']
Length: 7, dtype: str
```

```
No of rows:
(7,)
```

```
In [10]: categorical_insights("District_Name", df2)
```

```
Unique Values:
<StringArray>
[
    'NICOBARS', 'NORTH AND MIDDLE ANDAMAN',
    'SOUTH ANDAMANS', 'ANANTAPUR',
    'CHITTOOR', 'EAST GODAVARI',
    'GUNTUR', 'KADAPA',
    'KRISHNA', 'KURNOOL',
    ...
    'DINAJPUR UTTAR', 'HOOGHLY',
    'HOWRAH', 'JALPAIGURI',
    'MALDAH', 'MEDINIPUR EAST',
    'MEDINIPUR WEST', 'MURSHIDABAD',
    'NADIA', 'PURULIA']
Length: 646, dtype: str
```

```
No of rows:
(646,)
```

```
In [11]: categorical_insights("Season", df2)
```

```
Unique Values:
<StringArray>
['Kharif ', 'Whole Year ', 'Autumn ', 'Rabi ', 'Summer ',
 'Winter ']
Length: 6, dtype: str
```

```
No of rows:
(6,)
```

```
In [12]: categorical_insights("Crop", df2)
```

```
Unique Values:
<StringArray>
[
    'Arecanut', 'Other Kharif pulses', 'Rice',
    'Banana', 'Cashewnut', 'Coconut ',
    'Dry ginger', 'Sugarcane', 'Sweet potato',
    'Tapioca',
    ...
    'Ribed Guard', 'Yam', 'Apple',
    'Peach', 'Pear', 'Plums',
    'Litchi', 'Ber', 'Other Dry Fruit',
    'Jute & mesta']
Length: 124, dtype: str
```

```
No of rows:
(124,)
```

Insights of this Task

The **Dataset** contains columns with float data type and column string datatype. this datatypes support the dataset for the City AQI. we see a huge increase in the number of missing values in this dataset. i took categorical insights on column names such as **AQI_Bucket**.

The **Dataset** we see columns with float data type, one integer, and string data type data in it. This data set gives us insights on the crops that were grown, over the years in the area. the missing dataset has comparatively less missing values. i took categorical insights on column names such as **District_Name**, **Season**, and **Crop**.

Task — Missing Value Treatment

Requirements

For every column containing missing values:

- Decide whether to:
 - Remove the column
 - Remove affected rows
 - Impute missing values
- Justify why the chosen method is appropriate for that specific column

```
In [13]: def dataset_insights(df):  
  
    # Numeric Columns  
    numeric_df = df.select_dtypes(include='number')  
  
    # Categorical Columns  
    categorical_df = df.select_dtypes(exclude='number')  
  
    print("===== NUMERICAL DATA =====\n")  
  
    if not numeric_df.empty:  
  
        print("Mean:\n")  
        print(numeric_df.mean())  
  
        print("\nMedian:\n")  
        print(numeric_df.median())  
  
        print("\nMode:\n")  
        print(numeric_df.mode())  
  
        print("\nStandard Deviation:\n")  
        print(numeric_df.std())
```

```

print("\nVariance:\n")
print(numeric_df.var())

print("\nMinimum Values:\n")
print(numeric_df.min())

print("\nMaximum Values:\n")
print(numeric_df.max())

print("\nQuartiles:\n")
print(numeric_df.quantile([0.25, 0.5, 0.75]))

else:
    print("No numerical columns found.")

print("\n\n===== CATEGORICAL DATA =====\n")

if not categorical_df.empty:

    for col in categorical_df.columns:

        print(f"\nColumn: {col}")

        print("\nUnique Values:")
        print(categorical_df[col].unique())

        print("\nFrequency Count:")
        print(categorical_df[col].value_counts())

        print("\nMode:")
        print(categorical_df[col].mode())

        print("\nNumber of Unique Values:")
        print(categorical_df[col].nunique())

        print("\n-----")

else:
    print("No categorical columns found.")

```

In [14]: dataset_insights(df1)

===== NUMERICAL DATA =====

Mean:

PM2.5	67.450578
PM10	118.127103
NO	17.574730
NO2	28.560659
NOx	32.309123
NH3	23.483476
CO	2.248598
SO2	14.531977
O3	34.491430
Benzene	3.280840
Toluene	8.700972
Xylene	3.070128
AQI	166.463581

dtype: float64

Median:

PM2.5	48.57
PM10	95.68
NO	9.89
NO2	21.69
NOx	23.52
NH3	15.85
CO	0.89
SO2	9.16
O3	30.84
Benzene	1.07
Toluene	2.97
Xylene	0.98
AQI	118.00

dtype: float64

Mode:

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	\
0	11.0	94.0	5.93	10.58	0.0	6.29	0.0	5.74	16.48	0.0	0.0	

	Xylene	AQI
0	0.0	102.0

Standard Deviation:

PM2.5	64.661449
PM10	90.605110
NO	22.785846
NO2	24.474746
NOx	31.646011
NH3	25.684275
CO	6.962884
SO2	18.133775
O3	21.694928
Benzene	15.811136
Toluene	19.969164
Xylene	6.323247
AQI	140.696585

dtype: float64

Variance:

PM2.5	4181.103046
PM10	8209.285907
NO	519.194793
NO2	599.013182
NOx	1001.470008
NH3	659.681982
CO	48.481757
SO2	328.833790
O3	470.669909
Benzene	249.992035
Toluene	398.767497
Xylene	39.983458
AQI	19795.529057

dtype: float64

Minimum Values:

PM2.5	0.04
PM10	0.01
NO	0.02
NO2	0.01
NOx	0.00
NH3	0.01
CO	0.00
SO2	0.01
O3	0.01
Benzene	0.00
Toluene	0.00
Xylene	0.00
AQI	13.00

dtype: float64

Maximum Values:

PM2.5	949.99
PM10	1000.00
NO	390.68
NO2	362.21
NOx	467.63
NH3	352.89
CO	175.81
SO2	193.86
O3	257.73
Benzene	455.03
Toluene	454.85
Xylene	170.37
AQI	2049.00

dtype: float64

Quartiles:

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	\
0.25	28.82	56.255	5.63	11.75	12.8200	8.58	0.51	5.67	18.86	
0.50	48.57	95.680	9.89	21.69	23.5200	15.85	0.89	9.16	30.84	
0.75	80.59	149.745	19.95	37.62	40.1275	30.02	1.45	15.22	45.57	

Benzene	Toluene	Xylene	AQI
---------	---------	--------	-----

0.25	0.12	0.60	0.14	81.0
0.50	1.07	2.97	0.98	118.0
0.75	3.08	9.15	3.35	208.0

===== CATEGORICAL DATA =====

Column: City

Unique Values:

<StringArray>

```
[
    'Ahmedabad',      'Aizawl',      'Amaravati',
    'Amritsar',       'Bengaluru',   'Bhopal',
    'Brajrajnagar',   'Chandigarh',  'Chennai',
    'Coimbatore',     'Delhi',       'Ernakulam',
    'Gurugram',       'Guwahati',    'Hyderabad',
    'Jaipur',         'Jorapokhar',  'Kochi',
    'Kolkata',        'Lucknow',     'Mumbai',
    'Patna',          'Shillong',    'Talcher',
    'Thiruvananthapuram', 'Visakhapatnam']
```

Length: 26, dtype: str

Frequency Count:

City	
Ahmedabad	2009
Bengaluru	2009
Chennai	2009
Delhi	2009
Lucknow	2009
Mumbai	2009
Hyderabad	2006
Patna	1858
Gurugram	1679
Visakhapatnam	1462
Amritsar	1221
Jorapokhar	1169
Jaipur	1114
Thiruvananthapuram	1112
Amaravati	951
Brajrajnagar	938
Talcher	925
Kolkata	814
Guwahati	502
Coimbatore	386
Shillong	310
Chandigarh	304
Bhopal	289
Ernakulam	162
Kochi	162
Aizawl	113

Name: count, dtype: int64

Mode:

0	Ahmedabad
1	Bengaluru
2	Chennai
3	Delhi
4	Lucknow
5	Mumbai

Name: City, dtype: str

Number of Unique Values:
26

Column: Date

Unique Values:

<StringArray>

['2015-01-01', '2015-01-02', '2015-01-03', '2015-01-04', '2015-01-05',
'2015-01-06', '2015-01-07', '2015-01-08', '2015-01-09', '2015-01-10',
...
'2020-06-22', '2020-06-23', '2020-06-24', '2020-06-25', '2020-06-26',
'2020-06-27', '2020-06-28', '2020-06-29', '2020-06-30', '2020-07-01']

Length: 2009, dtype: str

Frequency Count:

Date

2020-03-11	26
2020-03-12	26
2020-03-13	26
2020-03-14	26
2020-03-15	26

	..
2015-05-30	7
2015-05-31	7
2015-01-01	6
2015-01-02	6
2015-01-03	6

Name: count, Length: 2009, dtype: int64

Mode:

0	2020-03-11
1	2020-03-12
2	2020-03-13
3	2020-03-14
4	2020-03-15

	...
108	2020-06-27
109	2020-06-28
110	2020-06-29
111	2020-06-30
112	2020-07-01

Name: Date, Length: 113, dtype: str

Number of Unique Values:
2009

Column: AQI_Bucket

Unique Values:

<StringArray>

[nan, 'Poor', 'Very Poor', 'Severe', 'Moderate', 'Satisfactory', 'Good']
Length: 7, dtype: str

Frequency Count:

```
AQI_Bucket
Moderate      8829
Satisfactory  8224
Poor          2781
Very Poor     2337
Good          1341
Severe        1338
Name: count, dtype: int64

Mode:
0    Moderate
Name: AQI_Bucket, dtype: str

Number of Unique Values:
6
```

```
In [15]: dataset_insights(df2)
```

===== NUMERICAL DATA =====

Mean:

```
Crop_Year      2005.643018
Area           12002.820864
Production     582503.442251
dtype: float64
```

Median:

```
Crop_Year      2006.0
Area           582.0
Production     729.0
dtype: float64
```

Mode:

```
   Crop_Year  Area  Production
0         2003    1.0          1.0
```

Standard Deviation:

```
Crop_Year      4.952164e+00
Area           5.052340e+04
Production     1.706581e+07
dtype: float64
```

Variance:

```
Crop_Year      2.452393e+01
Area           2.552614e+09
Production     2.912420e+14
dtype: float64
```

Minimum Values:

```
Crop_Year      1997.00
Area           0.04
Production     0.00
dtype: float64
```

Maximum Values:

```
Crop_Year      2.015000e+03
Area           8.580100e+06
Production     1.250800e+09
dtype: float64
```

Quartiles:

```
   Crop_Year  Area  Production
0.25    2002.0   80.0         88.0
0.50    2006.0  582.0        729.0
0.75    2010.0 4392.0       7023.0
```

===== CATEGORICAL DATA =====

Column: State_Name

Unique Values:

<StringArray>

```
['Andaman and Nicobar Islands', 'Andhra Pradesh',
  'Arunachal Pradesh', 'Assam',
  'Bihar', 'Chandigarh',
  'Chhattisgarh', 'Dadra and Nagar Haveli',
  'Goa', 'Gujarat',
  'Haryana', 'Himachal Pradesh',
  'Jammu and Kashmir', 'Jharkhand',
  'Karnataka', 'Kerala',
  'Madhya Pradesh', 'Maharashtra',
  'Manipur', 'Meghalaya',
  'Mizoram', 'Nagaland',
  'Odisha', 'Puducherry',
  'Punjab', 'Rajasthan',
  'Sikkim', 'Tamil Nadu',
  'Telangana', 'Tripura',
  'Uttar Pradesh', 'Uttarakhand',
  'West Bengal']
```

Length: 33, dtype: str

Frequency Count:

State_Name	
Uttar Pradesh	33306
Madhya Pradesh	22943
Karnataka	21122
Bihar	18885
Assam	14628
Odisha	13575
Tamil Nadu	13547
Maharashtra	12628
Rajasthan	12514
Chhattisgarh	10709
Andhra Pradesh	9628
West Bengal	9613
Gujarat	8436
Haryana	5875
Telangana	5649
Uttarakhand	4896
Kerala	4261
Nagaland	3906
Punjab	3173
Meghalaya	2867
Arunachal Pradesh	2546
Himachal Pradesh	2494
Jammu and Kashmir	1634
Tripura	1412
Manipur	1267
Jharkhand	1266
Mizoram	957
Puducherry	876
Sikkim	714
Dadra and Nagar Haveli	263
Goa	208
Andaman and Nicobar Islands	203
Chandigarh	90

Name: count, dtype: int64

```
Mode:
0    Uttar Pradesh
Name: State_Name, dtype: str
```

```
Number of Unique Values:
33
```

```
Column: District_Name
```

```
Unique Values:
```

```
<StringArray>
```

```
[
    'NICOBARS', 'NORTH AND MIDDLE ANDAMAN',
    'SOUTH ANDAMANS', 'ANANTAPUR',
    'CHITTOOR', 'EAST GODAVARI',
    'GUNTUR', 'KADAPA',
    'KRISHNA', 'KURNOOL',
    ...
    'DINAJPUR UTTAR', 'HOOGHLY',
    'HOWRAH', 'JALPAIGURI',
    'MALDAH', 'MEDINIPUR EAST',
    'MEDINIPUR WEST', 'MURSHIDABAD',
    'NADIA', 'PURULIA']
```

```
Length: 646, dtype: str
```

```
Frequency Count:
```

```
District_Name
```

```
BIJAPUR      945
TUMKUR       936
BELGAUM      925
HASSAN       895
BELLARY      887
```

```
...
HYDERABAD    8
KHUNTI       6
RAMGARH      6
NAMSAI       1
MUMBAI       1
```

```
Name: count, Length: 646, dtype: int64
```

```
Mode:
```

```
0    BIJAPUR
```

```
Name: District_Name, dtype: str
```

```
Number of Unique Values:
```

```
646
```

```
Column: Season
```

```
Unique Values:
```

```
<StringArray>
```

```
['Kharif ', 'Whole Year ', 'Autumn ', 'Rabi ', 'Summer ',
 'Winter ']
```

```
Length: 6, dtype: str
```

```
Frequency Count:
```

```
Season
```

```

Kharif          95951
Rabi            66987
Whole Year      57305
Summer          14841
Winter          6058
Autumn          4949
Name: count, dtype: int64

```

```

Mode:
0    Kharif
Name: Season, dtype: str

```

```

Number of Unique Values:
6

```

```

Column: Crop

```

```

Unique Values:
<StringArray>
[      'Arecanut', 'Other Kharif pulses',      'Rice',
      'Banana',      'Cashewnut',      'Coconut ',
      'Dry ginger',      'Sugarcane',      'Sweet potato',
      'Tapioca',
      ...
      'Ribed Guard',      'Yam',      'Apple',
      'Peach',      'Pear',      'Plums',
      'Litchi',      'Ber',      'Other Dry Fruit',
      'Jute & mesta']
Length: 124, dtype: str

```

```

Frequency Count:
Crop
Rice          15104
Maize         13947
Moong(Green Gram) 10318
Urad          9850
Sesamum       9046
      ...
Plums          6
Litchi         6
Apple          4
Peach          4
Other Dry Fruit 1
Name: count, Length: 124, dtype: int64

```

```

Mode:
0    Rice
Name: Crop, dtype: str

```

```

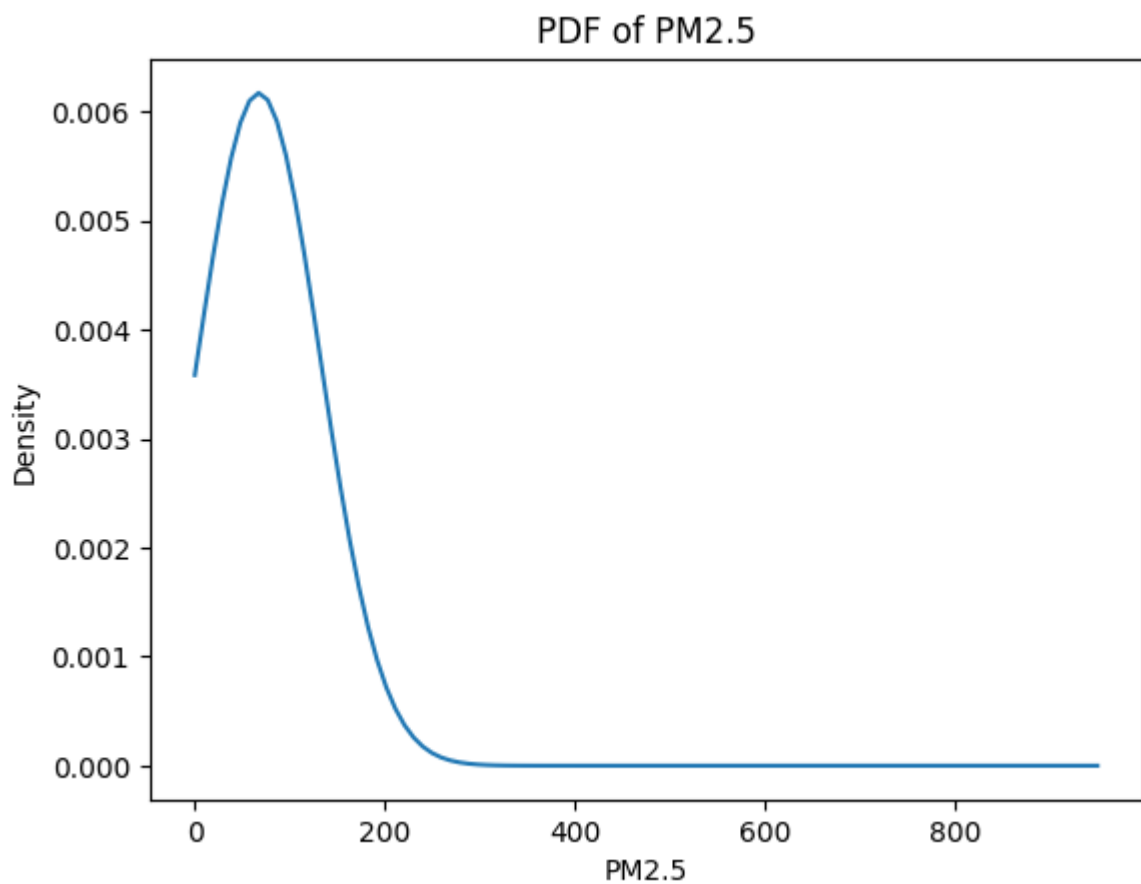
Number of Unique Values:
124

```

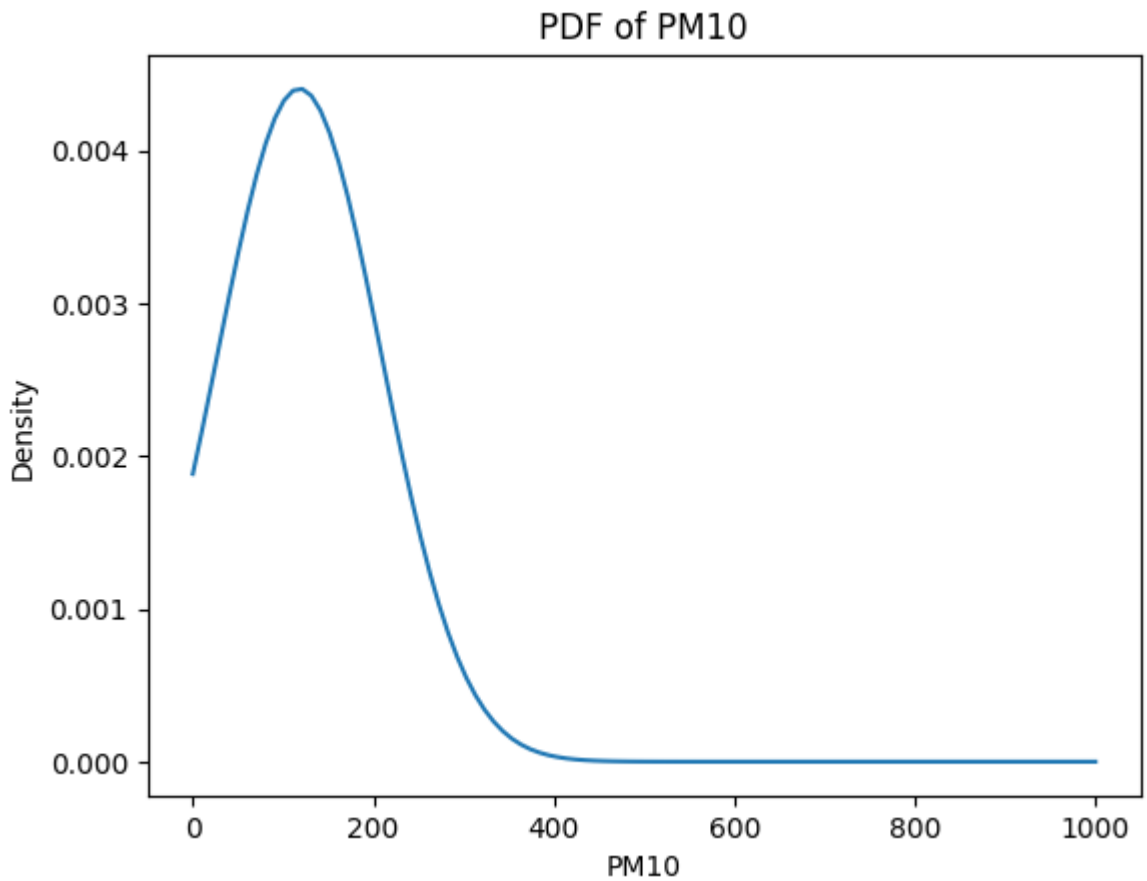
PDF (Probability Distribution Function)

```
In [16]: def pdf(column, df):  
    mean = df[column].mean()  
    std = df[column].std()  
  
    x = np.linspace(df[column].min(),  
                    df[column].max(),  
                    100)  
  
    pdf = norm.pdf(x, mean, std)  
  
    plt.plot(x, pdf)  
  
    plt.title(f"PDF of {column}")  
    plt.xlabel(f"{column}")  
    plt.ylabel("Density")  
  
    plt.show()
```

```
In [17]: pdf("PM2.5", df1)
```



```
In [18]: pdf("PM10", df1)
```

Missing Value Treatment Strategy

Dataset — city_day.csv

Column	Missing	Strategy	Justification
Xylene	(100 , 100) %	Drop column	Over 100 % missing. Xylene is a VOC not used in the CPCB AQI form making imputation unreliable at this scale.
PM	(27 , 27) %	Median imputation	Sensor data with heavy right-skew — median is more robust than mean.
NH	(20 , 20) %	Median imputation	Seasonal variation exists; median preserves the central tendency without outlier distortion.
Toluene	(100 , 100) %	Median imputation	Same reasoning as NH.
PM _{2.5} , NO ₂ , NO _x , CO, SO ₂ , O ₃ , Benzene	(100 , 100) %	Median imputation	All are pollutant sensor readings with skewed distributions observed above.
AQI	(0 , 0) %	Median per City	City-specific medians preserve local patterns better than a global median.
AQI_Bucket	(0 , 0) %	Derived from imputed AQI	AQI_Bucket is directly deterministic from using CPCB breakpoints.

Dataset — crop_production.csv

Column	Missing	Strategy	Justification
Production	(. %)	Drop rows	Only . % of , rows — negligibl Production cannot be reliably estimated without granular s weather context.

```
In [19]: print("Null counts BEFORE treatment (df1):")
print(df1.isna().sum())
print()

df1.drop(columns=['Xylene'], inplace=True)

for col in ['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene']:
    df1[col].fillna(df1[col].median(), inplace=True)

df1['AQI'] = df1.groupby('City')['AQI'].transform(lambda x: x.fillna(x.median()))

def aqi_to_bucket(aqi):
    if aqi <= 50: return 'Good'
    elif aqi <= 100: return 'Satisfactory'
    elif aqi <= 200: return 'Moderate'
    elif aqi <= 300: return 'Poor'
    elif aqi <= 400: return 'Very Poor'
    else: return 'Severe'

df1['AQI_Bucket'] = df1['AQI'].apply(aqi_to_bucket)

print("Null counts AFTER treatment (df1):")
print(df1.isna().sum())
```

Null counts BEFORE treatment (df1):

City	0
Date	0
PM2.5	4598
PM10	11140
NO	3582
NO2	3585
NOx	4185
NH3	10328
CO	2059
S02	3854
O3	4022
Benzene	5623
Toluene	8041
Xylene	18109
AQI	4681
AQI_Bucket	4681

dtype: int64

Null counts AFTER treatment (df1):

City	0
Date	0
PM2.5	4598
PM10	11140
NO	3582
NO2	3585
NOx	4185
NH3	10328
CO	2059
S02	3854
O3	4022
Benzene	5623
Toluene	8041
AQI	0
AQI_Bucket	0

dtype: int64

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being set on a copy of a DataFrame or Series through chained assignment using an inplace method.
```

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing `'df[col].method(value, inplace=True)'`, try using `'df.method({col: value}, inplace=True)'` instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using `'df[col] = df[col].method(value)'`.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being assigned to a copy of a DataFrame or Series through chained assignment using an inplace method.

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

/tmp/ipykernel_48681/3670036295.py:8: ChainedAssignmentError: A value is being assigned to a copy of a DataFrame or Series through chained assignment using an inplace method.

Such inplace method never works to update the original DataFrame or Series, because the intermediate object on which we are setting values always behaves as a copy (due to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' instead, to perform the operation inplace on the original object, or try to avoid an inplace operation using 'df[col] = df[col].method(value)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df1[col].fillna(df1[col].median(), inplace=True)
```

```
In [20]: print("Null counts BEFORE treatment (df2):")
print(df2.isna().sum())

df2.dropna(subset=['Production'], inplace=True)

print("\nNull counts AFTER treatment (df2):")
print(df2.isna().sum())
print(f"\nRemaining rows in df2: {len(df2):,}")
```

```
Null counts BEFORE treatment (df2):
State_Name      0
District_Name    0
Crop_Year        0
Season           0
Crop             0
Area            0
Production      3730
dtype: int64
```

```
Null counts AFTER treatment (df2):
State_Name      0
District_Name    0
Crop_Year        0
Season           0
Crop             0
Area            0
Production      0
dtype: int64
```

Remaining rows in df2: 242,361

Task — Data Cleaning and Standardization

Inconsistencies Found

Dataset (crop_production.csv):

- **State_Name** — 'Jammu and Kashmir ' and 'Telangana ' contain trailing white which will cause merge failures.
- **Season** — All six values have trailing spaces (e.g. 'Kharif ', 'Rabi ').
- **Crop** — Several entries have trailing whitespace (e.g. 'Coconut ', 'Dry ginger ').

Dataset (city_day.csv):

- No **State** column exists — cities must be mapped to their states to allow a future merge c

Fix applied: `.str.strip()` on all categorical columns in df ; a city → state dictionary ad
df .

```
In [21]: print("Before cleaning – State_Name unique values:")
print(sorted(df2['State_Name'].unique()))

for col in ['State_Name', 'Season', 'Crop', 'District_Name']:
    df2[col] = df2[col].str.strip()

print("\nAfter cleaning – State_Name unique values:")
print(sorted(df2['State_Name'].unique()))
print("\nSeason values after strip:", sorted(df2['Season'].unique()))
```

Before cleaning – State_Name unique values:

```
['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']
```

After cleaning – State_Name unique values:

```
['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir', 'Jharkhand', 'Karnataka', 'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']
```

Season values after strip: ['Autumn', 'Kharif', 'Rabi', 'Summer', 'Whole Year', 'Winter']

```
In [22]: city_to_state = {
    'Ahmedabad': 'Gujarat', 'Aizawl': 'Mizoram', 'Amaravati': 'Andhra Pradesh',
    'Amritsar': 'Punjab', 'Bengaluru': 'Karnataka', 'Bhopal': 'Madhya Pradesh',
    'Brajrajnagar': 'Odisha', 'Chandigarh': 'Chandigarh', 'Chennai': 'Tamil Nadu',
    'Coimbatore': 'Tamil Nadu', 'Delhi': 'Delhi', 'Ernakulam': 'Kerala',
    'Gurugram': 'Haryana', 'Guwahati': 'Assam', 'Hyderabad': 'Telangana',
    'Jaipur': 'Rajasthan', 'Jorapokhar': 'Jharkhand', 'Kochi': 'Kerala',
    'Kolkata': 'West Bengal', 'Lucknow': 'Uttar Pradesh', 'Mumbai': 'Maharashtra',
    'Patna': 'Bihar', 'Shillong': 'Meghalaya', 'Talcher': 'Odisha',
    'Thiruvananthapuram': 'Kerala', 'Visakhapatnam': 'Andhra Pradesh',
}

df1['State'] = df1['City'].map(city_to_state)
print(df1[['City', 'State']].drop_duplicates().sort_values('State').reset_
```


	City	State
0	Visakhapatnam	Andhra Pradesh
1	Amaravati	Andhra Pradesh
2	Guwahati	Assam
3	Patna	Bihar
4	Chandigarh	Chandigarh
5	Delhi	Delhi
6	Ahmedabad	Gujarat
7	Gurugram	Haryana
8	Jorapokhar	Jharkhand
9	Bengaluru	Karnataka
10	Kochi	Kerala
11	Ernakulam	Kerala
12	Thiruvananthapuram	Kerala
13	Bhopal	Madhya Pradesh
14	Mumbai	Maharashtra
15	Shillong	Meghalaya
16	Aizawl	Mizoram
17	Brajrajnagar	Odisha
18	Talcher	Odisha
19	Amritsar	Punjab
20	Jaipur	Rajasthan
21	Chennai	Tamil Nadu
22	Coimbatore	Tamil Nadu
23	Hyderabad	Telangana
24	Lucknow	Uttar Pradesh
25	Kolkata	West Bengal

```
In [23]: print(f"Duplicates in df1 before removal: {df1.duplicated().sum()}")
print(f"Duplicates in df2 before removal: {df2.duplicated().sum()}")

df1.drop_duplicates(inplace=True)
df2.drop_duplicates(inplace=True)

print(f"\ndf1 rows after deduplication: {len(df1):,}")
print(f"df2 rows after deduplication: {len(df2):,}")
```

Duplicates in df1 before removal: 0
 Duplicates in df2 before removal: 0

df1 rows after deduplication: 29,531
 df2 rows after deduplication: 242,361

Task Summary

Both datasets are now ready for merging:

- Whitespace stripped from all categorical columns in df
- Jammu and Kashmir and Telangana state names are now clean
- A State column added to df via city-to-state mapping
- Zero duplicates found in both datasets after treatment

Task — AQI Distribution Analysis

The pollution control board wants to understand:

- . Whether most cities are moderately polluted or pollution is concentrated in a few cities
- . Whether the average AQI is being distorted by extreme values

We will use a **histogram** to show the overall shape and clustering of values, and a **box plot** to highlight extreme readings.

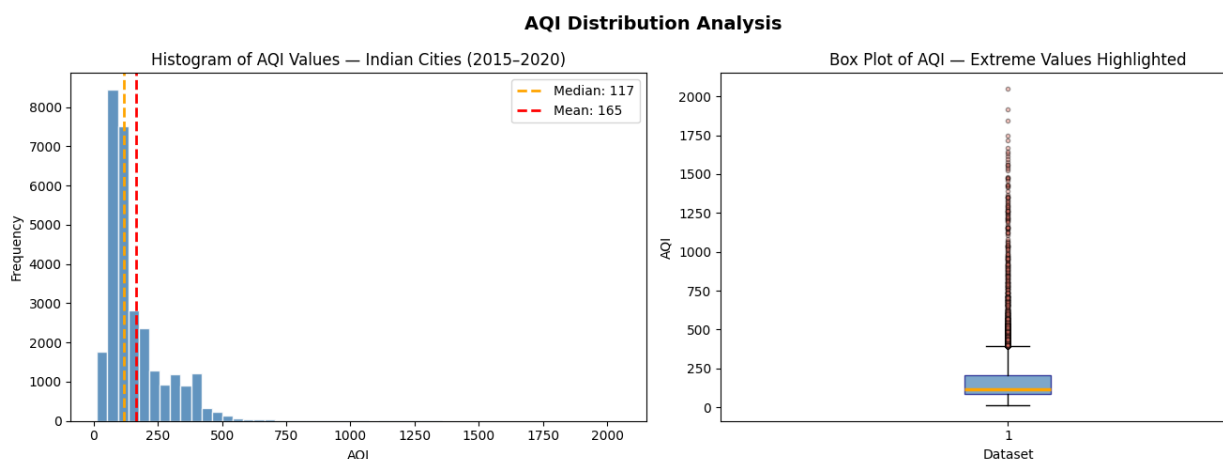
A histogram reveals the distribution shape (skewness, peaks) while a box plot directly surfaces the median, IQR, and outlier points — together they fully answer both board questions.

```
In [24]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].hist(df1['AQI'].dropna(), bins=50, color='steelblue', edgecolor='black')
axes[0].axvline(df1['AQI'].median(), color='orange', linestyle='--', linewidth=2,
                label=f"Median: {df1['AQI'].median():.0f}")
axes[0].axvline(df1['AQI'].mean(), color='red', linestyle='--', linewidth=2,
                label=f"Mean: {df1['AQI'].mean():.0f}")
axes[0].set_title('Histogram of AQI Values — Indian Cities (2015–2020)',
                  fontweight='bold')
axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')
axes[0].legend()

bp = axes[1].boxplot(df1['AQI'].dropna(), vert=True, patch_artist=True,
                    boxprops=dict(facecolor='steelblue', color='navy', alpha=0.5),
                    medianprops=dict(color='orange', linewidth=2.5),
                    flierprops=dict(marker='o', markerfacecolor='salmon', markersize=10))
axes[1].set_title('Box Plot of AQI — Extreme Values Highlighted', fontweight='bold')
axes[1].set_ylabel('AQI')
axes[1].set_xlabel('Dataset')

plt.suptitle('AQI Distribution Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Observations

Observation — Pollution is concentrated, not uniform:

The histogram shows a strong right-skew with the bulk of readings clustered below 250 (Moderate range). A long right tail extends toward very high AQI values, indicating that extreme pollution levels are present but less frequent.

is present in only a small fraction of city-days — most cities are in the Moderate-to-Satisfactory range most of the time.

Observation — Mean is inflated by extremes:

The mean AQI (~) sits noticeably above the median (~). This confirms the board's concern — the arithmetic mean is being pulled upward by extreme readings at the tail. The **median** is a **more representative measure** of typical air quality, as it is unaffected by the outlier events.

Task — Outlier Detection and Treatment

Method: Interquartile Range (IQR)

We use the **IQR method** because:

- It is non-parametric — no assumption of normality is needed (and we've seen AQI is right-skewed).
- It is robust to the very outliers it is trying to detect.
- The standard $1.5 \times \text{IQR}$ fence is widely used for environmental data.

Treatment: Winsorization (Clipping)

Rather than dropping rows (which loses valid date/city data), we **cap** outlier values at the IQR fences. This preserves the row count and all non-AQI columns while limiting distortion from unrealistically extreme readings.

```
In [25]: Q1 = df1['AQI'].quantile(0.25)
Q3 = df1['AQI'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

n_outliers = ((df1['AQI'] < lower_bound) | (df1['AQI'] > upper_bound)).sum()

print(f"Q1: {Q1:.2f} | Q3: {Q3:.2f} | IQR: {IQR:.2f}")
print(f"Lower fence: {lower_bound:.2f} | Upper fence: {upper_bound:.2f}")
print(f"\nOutliers detected: {n_outliers:,} ({100 * n_outliers / len(df1):.2f}%)")
```

```
Q1: 86.00 | Q3: 208.00 | IQR: 122.00
Lower fence: -97.00 | Upper fence: 391.00
```

```
Outliers detected: 1,465 (4.96% of data)
```

```
In [26]: df1['AQI_raw'] = df1['AQI'].copy()
df1['AQI'] = df1['AQI'].clip(lower=lower_bound, upper=upper_bound)

print("AQI Statistics — BEFORE Winsorization:")
print(df1['AQI_raw'].describe().round(2))
print("\nAQI Statistics — AFTER Winsorization:")
print(df1['AQI'].describe().round(2))
```

AQI Statistics – BEFORE Winsorization:

```
count    29531.00
mean      165.37
std       135.32
min       13.00
25%       86.00
50%      117.00
75%      208.00
max      2049.00
Name: AQI_raw, dtype: float64
```

AQI Statistics – AFTER Winsorization:

```
count    29531.00
mean      157.33
std       102.74
min       13.00
25%       86.00
50%      117.00
75%      208.00
max       391.00
Name: AQI, dtype: float64
```

```
In [27]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

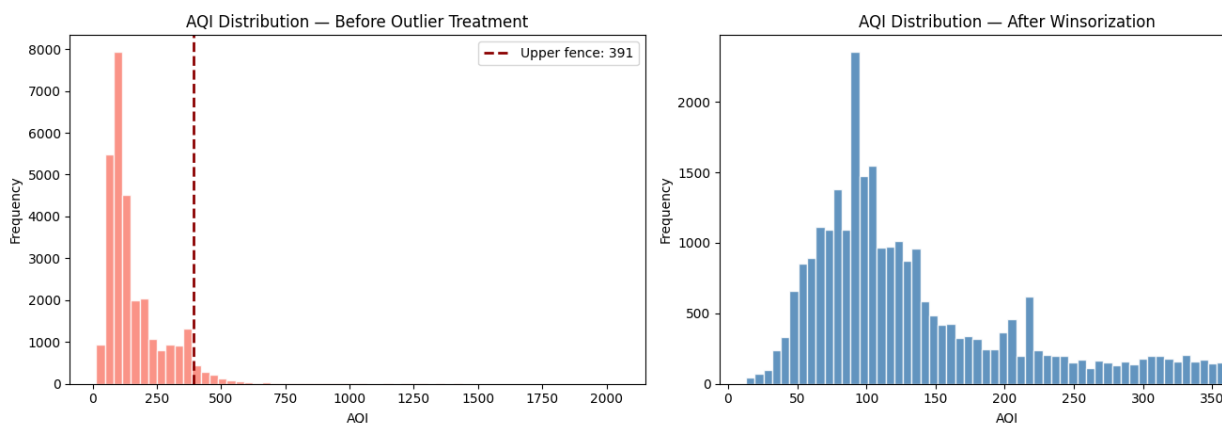
axes[0].hist(df1['AQI_raw'], bins=60, color='salmon', edgecolor='white',
axes[0].axvline(upper_bound, color='darkred', linestyle='--', linewidth=2
               label=f'Upper fence: {upper_bound:.0f}')
axes[0].set_title('AQI Distribution – Before Outlier Treatment', fontsize
axes[0].set_xlabel('AQI')
axes[0].set_ylabel('Frequency')
axes[0].legend()

axes[1].hist(df1['AQI'], bins=60, color='steelblue', edgecolor='white', a
axes[1].set_title('AQI Distribution – After Winsorization', fontsize=12)
axes[1].set_xlabel('AQI')
axes[1].set_ylabel('Frequency')

plt.suptitle('Outlier Treatment – Before vs After', fontsize=14, fontweig
plt.tight_layout()
plt.show()

df1.drop(columns=['AQI_raw'], inplace=True)
```

Outlier Treatment – Before vs After



Observations

- , **readings** (~ %) exceeded the upper IQR fence of ~ AQI and \ capped at that value.
- After Winsorization the distribution's right tail is sharply curtailed — the histogram becomes compact and the standard deviation drops significantly.
- No values fell below the lower fence (AQI is always positive and never near the calculated lo boundary).
- The capped dataset is now much more suitable for training ML models, as extreme values n dominate regression targets or inflate distance-based algorithms.